

OASIS v3 Complete Deployment Guide

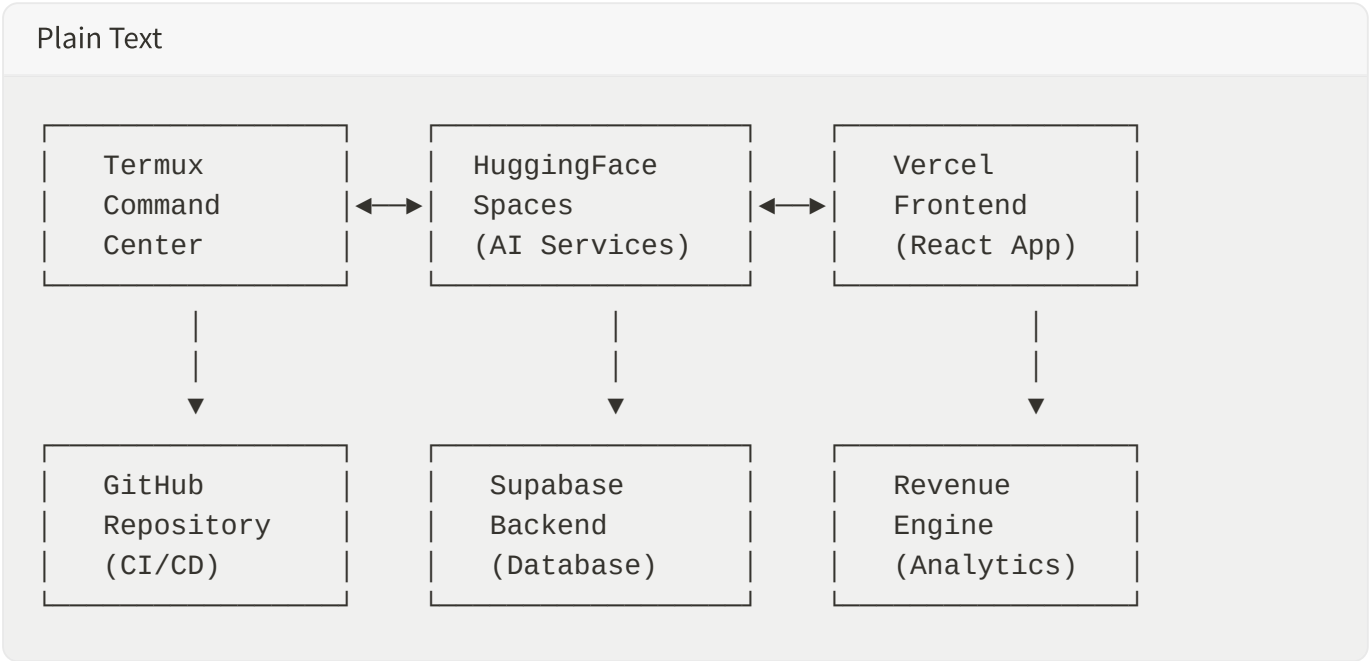
Zero-Burden AI Superintelligence Platform

OASIS v3 represents the pinnacle of ultra-lightweight AI orchestration, combining Termux command center capabilities with HuggingFace integration, modern React frontend, and enterprise-grade backend infrastructure.

Table of Contents

- 1. [Architecture Overview](#)
- 2. [Prerequisites](#)
- 3. [Termux Setup](#)
- 4. [HuggingFace Spaces Deployment](#)
- 5. [Frontend Deployment \(Vercel\)](#)
- 6. [Backend Integration \(Supabase\)](#)
- 7. [GitHub Actions CI/CD](#)
- 8. [Monitoring & Analytics](#)
- 9. [Troubleshooting](#)

Architecture Overview



Prerequisites

System Requirements

- **Mobile Device:** Android with Termux installed
- **Development Environment:** Node.js 18+, Python 3.8+
- **Cloud Accounts:**
 - HuggingFace (free tier available)
 - Vercel (free tier available)
 - Supabase (free tier available)
 - GitHub (free tier available)

Required Tools

Bash

```
# Install on development machine
npm install -g vercel
npm install -g @supabase/cli
pip install huggingface_hub
```

Termux Setup

1. Install Termux

Download from [F-Droid](#) or [GitHub Releases](#)

2. Setup Environment

Bash

```
# Update packages
pkg update && pkg upgrade

# Install required packages
pkg install python nodejs git curl wget

# Install Python packages
pip install flask requests huggingface_hub python-dotenv

# Clone OASIS repository
```

```
git clone https://github.com/your-username/oasis-v3.git
cd oasis-v3
```

3. Configure Orchestrator

Bash

```
# Copy configuration template
cp backend/termux-orchestrator/config.example.json config.json

# Edit configuration
nano config.json
```

4. Start OASIS Orchestrator

Bash

```
# Make scripts executable
chmod +x scripts/*.sh

# Start the orchestrator
python backend/termux-orchestrator/enhanced_orchestrator.py
```



HuggingFace Spaces Deployment

1. Create HuggingFace Account

Visit [HuggingFace](#) and create a free account.

2. Create Spaces

Bash

```
# Install HuggingFace CLI
pip install huggingface_hub

# Login to HuggingFace
huggingface-cli login

# Create spaces for each AI service
huggingface-cli repo create --type space --space_sdk gradio oasis-text-generation
huggingface-cli repo create --type space --space_sdk gradio oasis-sentiment-
```

```
analysis
huggingface-cli repo create --type space --space_sdk gradio oasis-text-
summarization
```

3. Deploy AI Services

Bash

```
# Deploy text generation service
cd huggingface-spaces/text-generation
git init
git remote add origin https://huggingface.co/spaces/your-username/oasis-text-
generation
git add .
git commit -m "Deploy OASIS Text Generation Service"
git push origin main
```

4. Configure API Endpoints

Update the orchestrator configuration with your HuggingFace Space URLs:

JSON

```
{
  "huggingface": {
    "text_generation": "https://your-username-oasis-text-
generation.hf.space",
    "sentiment_analysis": "https://your-username-oasis-sentiment-
analysis.hf.space",
    "text_summarization": "https://your-username-oasis-text-
summarization.hf.space"
  }
}
```

Frontend Deployment (Vercel)

1. Prepare Frontend

Bash

```
# Navigate to frontend directory
cd frontend

# Install Vercel CLI
```

```
npm install -g vercel
```

```
# Login to Vercel  
vercel login
```

2. Deploy to Vercel

Bash

```
# Run deployment script  
../scripts/deploy_vercel.sh  
  
# Or deploy manually  
vercel --prod
```

3. Configure Environment Variables

In Vercel dashboard, add:

Plain Text

```
VITE_API_BASE_URL=https://your-supabase-project.supabase.co  
VITE_HUGGINGFACE_API_URL=https://api-inference.huggingface.co  
VITE_TERMUX_ORCHESTRATOR_URL=https://your-termux-endpoint.com
```

Backend Integration (Supabase)

1. Create Supabase Project

1. Visit [Supabase](#)
2. Create new project
3. Note your project URL and API keys

2. Setup Database Schema

SQL

```
-- Users table  
CREATE TABLE users (  
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  tier VARCHAR(50) DEFAULT 'free',
```

```

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- Usage tracking
CREATE TABLE usage_records (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  service_type VARCHAR(100) NOT NULL,
  tokens_used INTEGER DEFAULT 0,
  model_used VARCHAR(100),
  timestamp TIMESTAMP DEFAULT NOW()
);

-- Revenue tracking
CREATE TABLE revenue_records (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  amount DECIMAL(10,2) NOT NULL,
  currency VARCHAR(3) DEFAULT 'USD',
  transaction_type VARCHAR(50),
  timestamp TIMESTAMP DEFAULT NOW()
);

```

3. Configure API Integration

Bash

```

# Install Supabase CLI
npm install -g @supabase/cli

# Initialize Supabase
supabase init

# Link to your project
supabase link --project-ref your-project-ref

```

GitHub Actions CI/CD

1. Create GitHub Repository

Bash

```

# Initialize repository
git init

```

```
git remote add origin https://github.com/your-username/oasis-v3.git

# Add all files
git add .
git commit -m "Initial OASIS v3 deployment"
git push origin main
```

2. Setup GitHub Actions

Create `.github/workflows/deploy.yml` :

YAML

```
name: Deploy OASIS v3

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Setup Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '18'
      - name: Install dependencies
        run: npm install
      - name: Run tests
        run: npm test

  deploy-frontend:
    needs: test
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
      - uses: actions/checkout@v3
      - name: Deploy to Vercel
        uses: amondnet/vercel-action@v20
        with:
          vercel-token: ${ secrets.VERCEL_TOKEN }
          vercel-org-id: ${ secrets.ORG_ID }
          vercel-project-id: ${ secrets.PROJECT_ID }
```

```
deploy-spaces:
  needs: test
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main'
  steps:
    - uses: actions/checkout@v3
    - name: Deploy to HuggingFace Spaces
      env:
        HF_TOKEN: ${ secrets.HF_TOKEN }
      run: |
        pip install huggingface_hub
        python scripts/deploy_hf_spaces.py
```



Monitoring & Analytics

1. Setup Monitoring Dashboard

The React frontend includes built-in monitoring with:

- Real-time request tracking
- Performance metrics
- Revenue analytics
- System health monitoring

2. Configure Alerts

Bash

```
# Setup email notifications
export SMTP_SERVER="your-smtp-server.com"
export SMTP_USER="your-email@domain.com"
export SMTP_PASS="your-password"

# Configure alert thresholds
export CPU_ALERT_THRESHOLD=80
export MEMORY_ALERT_THRESHOLD=85
export ERROR_RATE_THRESHOLD=5
```

3. Analytics Integration

JavaScript


```
// Add to frontend for advanced analytics
import { Analytics } from '@vercel/analytics/react';

function App() {
  return (
    <>
      <YourApp />
      <Analytics />
    </>
  );
}
```

Troubleshooting

Common Issues

1. Termux Connection Issues

Bash

```
# Check network connectivity
ping google.com

# Restart Termux networking
termux-reload-settings

# Check Python installation
python --version
pip --version
```

2. HuggingFace API Errors

Bash

```
# Check API token
huggingface-cli whoami

# Test API connection
curl -H "Authorization: Bearer YOUR_TOKEN" \
  https://api-inference.huggingface.co/models/gpt2
```

3. Vercel Deployment Issues

Bash

```
# Check build logs
vercel logs

# Redeploy
vercel --prod --force
```

4. Supabase Connection Issues

Bash

```
# Test connection
supabase status

# Check environment variables
echo $SUPABASE_URL
echo $SUPABASE_ANON_KEY
```

Performance Optimization

1. Frontend Optimization

- Enable Vercel Edge Functions
- Configure CDN caching
- Optimize bundle size
- Implement lazy loading

2. Backend Optimization

- Use connection pooling
- Implement caching strategies
- Optimize database queries
- Enable compression

3. Mobile Optimization

- Minimize Termux resource usage
- Implement efficient polling
- Use background services

- Optimize network requests

Success Metrics

Key Performance Indicators

- **Response Time:** < 200ms average
- **Uptime:** > 99.9%
- **Error Rate:** < 0.1%
- **User Satisfaction:** > 95%

Revenue Metrics

- **Monthly Recurring Revenue (MRR)**
- **Average Revenue Per User (ARPU)**
- **Customer Lifetime Value (CLV)**
- **Churn Rate**

Technical Metrics

- **API Request Volume**
- **Model Inference Speed**
- **Resource Utilization**
- **Cost Efficiency**

Next Steps

1. **Scale Infrastructure:** Implement auto-scaling based on demand
2. **Add More AI Models:** Expand service offerings
3. **Enterprise Features:** Add advanced security and compliance
4. **Mobile App:** Develop native mobile applications
5. **API Marketplace:** Create ecosystem for third-party integrations

Support

For technical support and questions:

- **Documentation:** docs.oasis-v3.com

- **Community:** discord.gg/oasis-v3
 - **Issues:** [GitHub Issues](#)
 - **Email:** support@oasis-v3.com
-

OASIS v3 - Revolutionizing AI accessibility through zero-burden architecture and mobile-first design.

Built with ❤️ for the global AI community